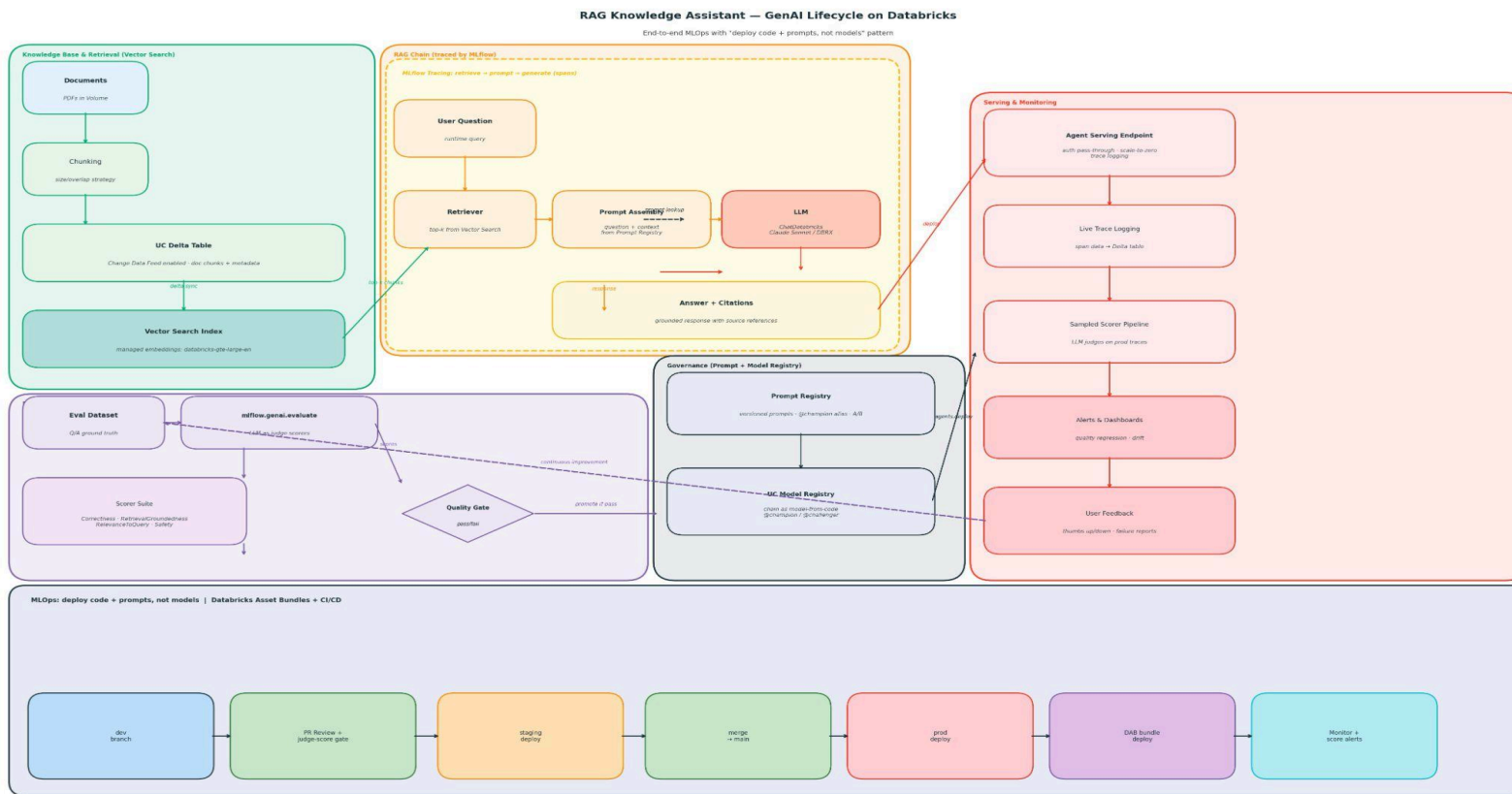


Databricks MLflow — GenAI (RAG) Lifecycle

Dev to Prod - A practical reference: a Retrieval-Augmented knowledge assistant, end to end



Databricks MLflow — GenAI (RAG) Lifecycle.....	1
Dev to Prod - A practical reference: a Retrieval-Augmented knowledge assistant, end to end.....	1
0. The mental model (read this first).....	4
1. Environment setup.....	5
Project skeleton.....	5
✅ Best practices.....	5
2. Knowledge base & retrieval (Vector Search).....	6
2.1 Chunk and store documents in a UC table.....	6
2.2 Create a Vector Search index.....	6
✅ Best practices.....	7
✅ Best practices.....	9
4. Tracing — observability for LLM apps.....	10
✅ Best practices.....	10
5. Evaluation — the heart of GenAI quality.....	11
5.1 A curated evaluation dataset.....	11
5.2 Score with LLM-as-judge scorers.....	11
The GenAI metric map.....	12
5.3 Human feedback (when judges aren't enough).....	12
✅ Best practices.....	12
6. Prompt & model registry (Unity Catalog).....	13
6.1 Prompt registry — version prompts like code.....	13
6.2 Register the chain as a UC model.....	13
✅ Best practices.....	14
7. Serving (Agent / Model Serving).....	15
✅ Best practices.....	15
8. Production monitoring.....	16
What to watch.....	16
✅ Best practices.....	17

9.1 The eval gate (evaluate.py).....	18
9.2 Asset Bundle + CI/CD.....	19
✓ Best practices.....	19
10. One-page lifecycle checklist.....	20
11. Common pitfalls (and the fix).....	21
Glossary (fast reference).....	21

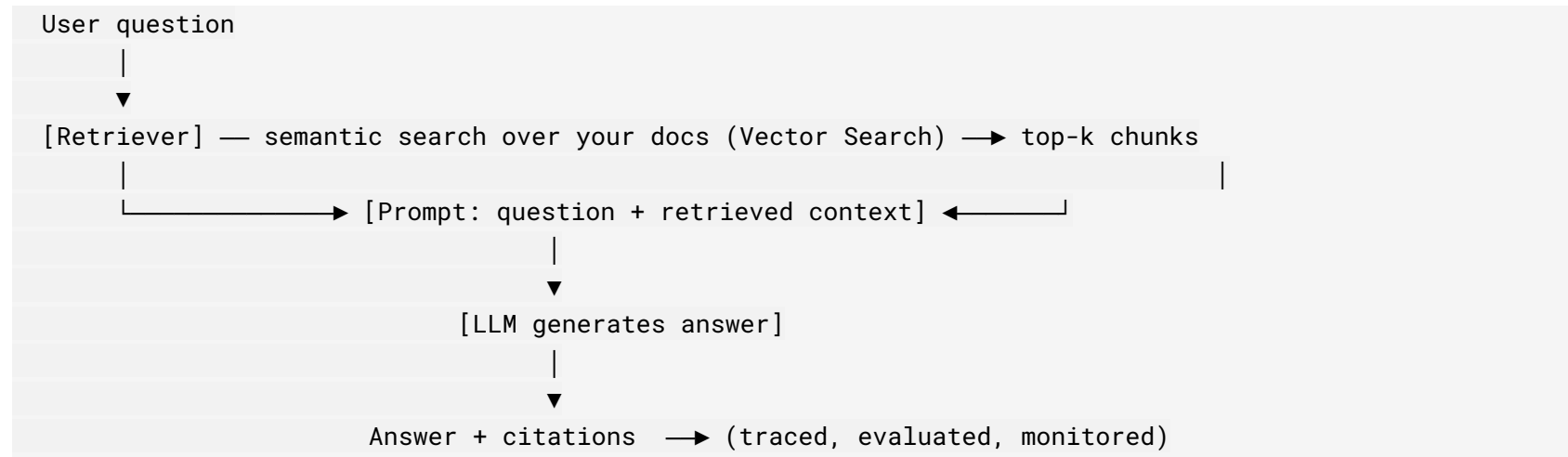
How to read this doc. Section floor the GenAI lifecycle:

Setup → Knowledge Base & Retrieval → Build the RAG Chain → Tracing → Evaluation → Prompt & Model Registry → Serving → Monitoring → MLOps Automation.

Each section has a **why it matters** callout, **key code**, and a ✓ **Best practices** list. Beginners read top-to-bottom; experienced readers jump to any section.

0. The mental model (read this first)

A RAG assistant answers questions using **your** documents, not just the LLM's training data:



Three ideas make the rest click:

1. **RAG has two failure surfaces.** *Retrieval* (did we fetch the right context?) and *generation* (did the LLM answer faithfully from that context?). You must be able to debug them separately.
2. **You can't grade with accuracy.** There's no single correct string. You evaluate with **traces** (the full step-by-step record of a request) scored by **LLM-as-judge** scorers (correctness, groundedness, relevance) plus optional human labels.
3. **Same MLOps spine as classic ML.** MLflow is still the system of record; Unity Catalog still governs; you still **deploy code, not models** — the "model" is now your chain/agent code + a versioned prompt.

1. Environment setup

Why it matters: GenAI apps have more moving parts (LLM endpoint, vector index, prompt, chain code). Pinning them is what makes a result reproducible.

Project skeleton

```
rag-assistant/  
├─ databricks.yml          # Asset Bundle: jobs, targets (dev/staging/prod)  
├─ requirements.txt        # mlflow>=3, databricks-vectorsearch, langchain, ...  
├─ src/  
│   ├─ ingest.py          # docs -> chunks -> vector index  
│   ├─ chain.py           # the RAG chain (logged as "model from code")  
│   └─ evaluate.py        # offline eval with LLM judges + gate  
├─ prompts/               # versioned prompt templates  
├─ data/eval/             # curated eval dataset (question + expected facts)  
└─ tests/
```

✓ Best practices

- Use **MLflow 3** (mlflow>=3) — it has native GenAI tracing, the prompt registry, and GenAI scorers.
- Pin the **LLM** and **embedding** model/endpoint names in config, not inline — you'll want to swap and A/B them.
- Keep the **eval dataset in Git/UC** from day one. In GenAI, the eval set *is* your test suite.

2. Knowledge base & retrieval (Vector Search)

Why it matters: RAG answer quality is capped by retrieval quality. If the right chunk isn't retrieved, no LLM can answer correctly. Chunking + embeddings + the index are where quality is won or lost.

2.1 Chunk and store documents in a UC table

```
from pyspark.sql import functions as F

docs = spark.read.text("/Volumes/dev/kb/landing/*.md", wholetext=True)

# Simple chunking; tune size/overlap to your content (see best practices)
chunks = (docs
    .withColumn("chunk", F.explode(chunk_udf(F.col("value")))) # your splitter
    .withColumn("id", F.monotonically_increasing_id())
    .select("id", "chunk"))

chunks.write.mode("overwrite").option("delta.enableChangeDataFeed", "true") \
    .saveAsTable("dev.kb.doc_chunks") # CDF lets the index sync incrementally
```

2.2 Create a Vector Search index

Why managed embeddings? Databricks Vector Search can compute and **keep embeddings in sync** with the source Delta table automatically (Change Data Feed). New/edited docs flow to the index without a manual re-embed job.

```
from databricks.vector_search.client import VectorSearchClient
vsc = VectorSearchClient()

vsc.create_delta_sync_index(
    endpoint_name="kb-endpoint",
    index_name="dev.kb.doc_index",
```

```
source_table_name="dev.kb.doc_chunks",  
pipeline_type="TRIGGERED",           # or CONTINUOUS for near-real-time  
primary_key="id",  
embedding_source_column="chunk",  
embedding_model_endpoint_name="databricks-gte-large-en",  # managed embeddings  
)
```

✓ Best practices

- **Chunking is a hyperparameter.** Too large → diluted context; too small → lost meaning. Start ~500–1000 tokens with ~10–15% overlap, then *measure* with retrieval eval (§5).
- Keep **metadata** (source URL, title, section) alongside each chunk so you can **cite** and filter.
- Enable **Change Data Feed** and delta-sync so the index stays fresh without bespoke pipelines.
- Govern the index and source table in **Unity Catalog** — same permissions/lineage as everything else.

3. Build the RAG chain

Why it matters: The chain is your deployable unit — the equivalent of the "model" in classic ML. Logging it as **code** (not a pickled object) is what keeps it reproducible and portable across environments.

```
# src/chain.py
import mlflow
from databricks_langchain import ChatDatabricks, DatabricksVectorSearch
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough

mlflow.langchain.autolog() # auto-traces every retrieval + LLM call

retriever = DatabricksVectorSearch(index_name="dev.kb.doc_index").as_retriever(search_kwargs={"k": 4})
llm = ChatDatabricks(endpoint="databricks-claude-sonnet-4") # served foundation model

prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer ONLY from the context. If unknown, say so. Cite sources.\n\nContext:\n{context}"),
    ("user", "{question}"),
])

chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt | llm
)

mlflow.models.set_model(chain) # marks this file as the deployable model ("model from code")
```

"Model from code": MLflow logs the *file* `chain.py` as the model, not a serialized object. On load it re-executes the code, so there are no pickle/version headaches and the chain is fully transparent and diffable in Git.

✓ Best practices

- Put a **strict system prompt**: answer only from context, admit uncertainty, cite sources. This is your first defense against hallucination.
- Externalize the prompt into the **prompt registry** (§6) so you can version and A/B it independently of code.
- Keep k (retrieved chunks) configurable; it's a quality/cost/latency knob you'll tune with eval.

4. Tracing — observability for LLM apps

Why it matters: An LLM answer is the end of a multi-step process (retrieve → assemble prompt → generate). When an answer is wrong, a **trace** tells you *which step* failed — did retrieval miss, or did the LLM ignore good context? This is the single most useful GenAI debugging tool.

```
import mlflow
mlflow.langchain.autolog()          # traces LangChain steps automatically

# For custom Python functions, decorate to add spans:
@mlflow.trace
def rerank(chunks): ...
```

A trace captures each **span**: retriever inputs/outputs (which chunks, similarity scores), the exact prompt sent, tokens, latency, and the final answer. Traces are captured in **dev** (debugging), during **evaluation** (the object judges score), and in **production** (monitoring) — the same primitive across the whole lifecycle.

✓ Best practices

- Turn on **autolog** first; add `@mlflow.trace` on custom steps (rerankers, guardrails, tools).
- Log **retrieval scores** into the span — most "bad answers" are actually "bad retrieval."
- Keep tracing **on in production** (sampled if volume is high) — it's what powers monitoring in §8.

5. Evaluation — the heart of GenAI quality

Why it matters: Without labels, "seems fine" is not a quality bar. MLflow's GenAI evaluation gives you **repeatable, quantitative** scores so you can compare prompt/chunking/model changes objectively — the same role `mlflow.evaluate` plays for classic ML.

5.1 A curated evaluation dataset

Small and high-quality beats large and noisy. 20–50 representative questions with expected facts/sources is a strong start.

```
eval_data = [
    {"inputs": {"question": "How do I reset my password?"},
      "expectations": {"expected_facts": ["Settings > Security", "reset link emailed"]}},
    {"inputs": {"question": "What is the refund window?"},
      "expectations": {"expected_facts": ["30 days", "original payment method"]}},
]
```

5.2 Score with LLM-as-judge scorers

```
import mlflow
from mlflow.genai.scorers import Correctness, RelevanceToQuery, RetrievalGroundedness, Safety

results = mlflow.genai.evaluate(
    data=eval_data,
    predict_fn=lambda question: chain.invoke(question),
    scorers=[
        Correctness(),          # does the answer match expected_facts?
        RelevanceToQuery(),     # does it actually address the question?
        RetrievalGroundedness(), # is the answer supported by retrieved context? (anti-hallucination)
        Safety(),               # toxic / unsafe content?
    ],
)
```

```
)  
# results.metrics -> aggregate scores; per-row traces show WHY each scored as it did
```

The GenAI metric map

Scorer	Failure it catches	Which half of RAG
RetrievalGroundedness	Hallucination (answer not supported by context)	Generation
RelevanceToQuery	Off-topic / evasive answers	Generation
Correctness	Wrong facts vs. ground truth	Both
Retrieval recall/precision	Right chunk not retrieved	Retrieval
Safety / guardrails	Toxic, PII, unsafe output	Generation

5.3 Human feedback (when judges aren't enough)

Use MLflow **Review Apps** / **labeling sessions** to collect expert judgments, then use them to **calibrate** or even fine-tune your LLM judges. Human labels are the ground truth your automated judges approximate.

Best practices

- **Separate retrieval and generation eval.** If groundedness is high but correctness is low, your retriever is missing the right chunks — fix retrieval, not the prompt.
- Version the **eval dataset**; grow it by adding every production failure you find (turn incidents into test cases).
- **Validate the judges:** spot-check LLM-judge scores against human labels so you trust the metric.
- Gate deploys on eval scores in CI (§9) — the GenAI equivalent of a passing test suite.

6. Prompt & model registry (Unity Catalog)

Why it matters: The prompt is often what you iterate on *most*, and it changes behavior as much as code. Versioning it separately — and registering the chain in UC — gives you a governed, rollback-able hand-off point.

6.1 Prompt registry — version prompts like code

```
import mlflow

# Register a new prompt version
mlflow.genai.register_prompt(
    name="dev.kb.rag_system_prompt",
    template="Answer ONLY from the context. If unknown, say so. Cite sources.\n\nContext:\n{{context}}",
    commit_message="Add explicit citation + refusal instruction",
)

# Load a specific version/alias in the chain
p = mlflow.genai.load_prompt("prompts:/dev.kb.rag_system_prompt@production")
```

6.2 Register the chain as a UC model

```
import mlflow

mlflow.set_registry_uri("databricks-uc")

with mlflow.start_run():
    info = mlflow.langchain.log_model(
        lc_model="src/chain.py",          # model-from-code
        artifact_path="chain",
        registered_model_name="dev.kb.rag_assistant",
    )
```

```
from mlflow.tracking import MlflowClient
MlflowClient().set_registered_model_alias("dev.kb.rag_assistant", "champion",
    info.registered_model_version)
```

✓ Best practices

- Version prompts in the **prompt registry**; reference them by **alias** (@production) so a prompt fix is a pointer move, not a redeploy.
- Register the chain in **UC**; use @champion/@challenger aliases exactly like classic ML.
- Tag each model version with its **eval scores + git sha** for audit and rollback.

7. Serving (Agent / Model Serving)

Why it matters: Serving turns your chain into an API your app can call — with the LLM, retriever, tracing, and auth wired together and scaled for you.

```
from databricks import agents
```

```
# Deploys the registered chain to a scalable, authenticated endpoint
agents.deploy(
    model_name="prod.kb.rag_assistant",
    model_version=info.registered_model_version,    # or serve by @champion alias
    scale_to_zero=True,
)
```

What you get out of the box: a REST endpoint, **automatic credential pass-through** to the vector index and LLM, **inference/trace logging** to a UC table, and an optional **Review App** for stakeholders to try it and leave feedback.

Best practices

- Serve by **alias** so prompt/chain promotions don't require redeploying the endpoint.
- Enable **scale-to-zero** for cost control on spiky/low traffic.
- Keep **trace logging on** — production traces feed monitoring (§8) and grow your eval set (§5).
- Put **guardrails** (safety scorer, PII filter) in the chain, not just the app, so every caller is protected.

8. Production monitoring

Why it matters: GenAI quality drifts too — your docs change, users ask new kinds of questions, the base model gets updated. Monitoring runs your **evaluation scorers on live traffic** so you catch regressions without waiting for complaints.

```
import mlflow
from mlflow.genai.scorers import RetrievalGroundedness, RelevanceToQuery, Safety

# Run scorers continuously on a sample of production traces
mlflow.genai.create_monitor(
    endpoint="prod-rag-assistant",
    scorers=[RetrievalGroundedness(), RelevanceToQuery(), Safety()],
    sampling_rate=0.1,      # score 10% of live traffic
)
```

What to watch

Signal	Question	Source
Groundedness trend	Are we starting to hallucinate more?	Judge on live traces
Relevance trend	Are answers drifting off-topic?	Judge on live traces
Retrieval quality	Are we still fetching good chunks?	Retrieval spans + scores
User feedback	Thumbs down / escalations	App feedback → traces
Cost & latency	Tokens, \$/query, p95 latency	Endpoint metrics
Safety	Unsafe outputs slipping through	Safety scorer

✓ Best practices

- **Sample** production traffic for judge scoring to control cost; alert on score drops.
- Capture **user feedback** (👍/👎, escalation) and link it to the trace — the richest signal you have.
- Every production failure becomes a new **eval case** — the loop that keeps quality rising.
- Watch **cost/latency** alongside quality; a "better" prompt that doubles tokens may not be worth it.

9. MLOps automation: "deploy code, not models" (GenAI edition)

Why it matters: Same philosophy as classic ML — promote reviewed **code + prompts** through dev → staging → prod, with an **eval gate** instead of an accuracy gate. Nothing fragile gets hand-copied.

dev		staging		prod
—		—		—
iterate on chain + prompt + index	—PR→	CI runs mlflow.genai .evaluate() on eval set gate on judge scores	—merge→	deploy chain, register @champion, serve endpoint, monitor live traces

9.1 The eval gate (evaluate.py)

```
# src/evaluate.py - fails CI if quality regresses
import mlflow, sys
from mlflow.genai.scorers import Correctness, RetrievalGroundedness

res = mlflow.genai.evaluate(data=eval_data, predict_fn=chain.invoke,
                           scorers=[Correctness(), RetrievalGroundedness()])

groundedness = res.metrics["retrieval-groundedness/mean"]
correctness = res.metrics["correctness/mean"]

THRESHOLDS = {"groundedness": 0.90, "correctness": 0.80}
if groundedness < THRESHOLDS["groundedness"] or correctness < THRESHOLDS["correctness"]:
    print(f"❌ Quality gate failed: grounded={groundedness:.2f}, correct={correctness:.2f}")
    sys.exit(1)      # blocks the deploy
print(f"✅ Quality gate passed")
```

9.2 Asset Bundle + CI/CD

Identical structure to the classic-ML doc: a `databricks.yml` with `dev/staging/prod` targets and `${var.catalog}` substitution, deployed by `databricks bundle deploy -t <target>` from GitHub Actions. The **only** difference is the pipeline task runs `mlflow.genai.evaluate` and the gate above instead of a classifier metric.

```
# .github/workflows/deploy.yml (essentials)
on: {pull_request: {branches: [main]}, push: {branches: [main]}}
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main
      - run: databricks bundle validate -t staging
      - run: python src/evaluate.py          # LLM-judge quality gate
      - if: github.ref == 'refs/heads/main'
        run: databricks bundle deploy -t prod # promote code + prompt to prod
```

✓ Best practices

- **Everything as code:** chain, prompts, index config, jobs, and thresholds live in Git + the bundle.
- Gate deploys on **judge scores**, not vibes; treat the eval set as your regression suite.
- Prod runs as a **service principal**; secrets in **Databricks secret scopes**.
- Promote via **aliases** (`@champion`); roll back by moving the pointer to the previous version.

10. One-page lifecycle checklist

Stage	Do this	The "why" in one line
Setup	MLflow 3, pinned LLM/embeddings, eval set in Git	Reproducibility from day one
Knowledge base	Chunk to UC table; delta-sync Vector Search index	Fresh, governed retrieval
Chain	Model-from-code, strict cite/refuse prompt	Transparent, portable, safer
Trace	autolog() + @mlflow.trace on custom steps	See which step failed
Evaluate	mlflow.genai.evaluate + LLM judges	Quantitative quality, no labels needed
Registry	Prompt registry + UC model, @champion alias	Governed, rollback-able hand-off
Serve	agents.deploy, alias-based, scale-to-zero	Scalable API with auth + tracing
Monitor	Scorers on sampled live traces + alerts	Catch drift and hallucination
Automate	Bundles + CI/CD + judge-score gate	Deploy code, not models

11. Common pitfalls (and the fix)

- **Blaming the LLM for bad answers** → check the trace; it's usually **retrieval**. Fix chunking/k/index first.
 - **No eval set** → you can't tell if a change helped. Curate 20–50 cases before tuning anything.
 - **Trusting LLM judges blindly** → validate them against human labels periodically.
 - **Prompt changes with no versioning** → use the prompt registry; a prompt is as impactful as code.
 - **Hallucination in prod** → enforce groundedness in the system prompt *and* gate on RetrievalGroundedness.
 - **Stale knowledge base** → delta-sync the index with Change Data Feed so docs stay current.
 - **Ignoring cost/latency** → monitor tokens and p95; a better answer that's 3× the cost may not ship.
-

Glossary (fast reference)

- **RAG** — Retrieval-Augmented Generation: ground LLM answers in retrieved documents.
- **Vector Search** — Databricks' managed semantic index; can auto-sync embeddings from a Delta table.
- **Chunk** — a passage of a document that gets embedded and retrieved.
- **Trace / span** — the recorded step-by-step execution of a request; a span is one step (retrieve, generate).
- **LLM-as-judge** — using an LLM to score outputs (correctness, groundedness, relevance, safety).
- **Groundedness** — whether an answer is actually supported by the retrieved context (anti-hallucination).
- **Model-from-code** — logging the chain's source file as the model instead of a serialized object.
- **Prompt registry** — versioned, aliasable store for prompt templates in Unity Catalog.
- **Agent serving** — Databricks serving for chains/agents with auth pass-through and trace logging.